RECONIQ: AN AGENTIC FINANCIAL RECONCILIATION SYSTEM

Project Phase-I Report submitted to Osmania University, Hyderabad, in partial fulfillment of the requirement for the award of the degree of

Bachelor of Engineering

In

Information Technology

By

Mr. Mohammed Sarwar Khan	**(160923737142)**
Mr. Syed Mohd Hifzan	**(160923737166)**
Mr. Md Mubashiruddin	**(160923737167)**

Under the Guidance of

Ms. M Shilpa

Assistant Professor



DEPARTMENT OF INFORMATION TECHNOLOGY

LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

(UGC Autonomous)

(Affiliated to Osmania University) Himayathsagar, Hyderabad,500091

A.Y. 2026-2027



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY
(UGC Autonomous)
(Affiliated to Osmania University)
Himayathsagar, Hyderabad, 500091
Department of Information Technology

CERTIFICATE

This is to certify that the Project Phase-I report entitled **ReconIQ: An Agentic Financial Reconciliation System** being submitted by **Mohammed Sarwar Khan (160923737142), Syed Mohd Hifzan (160923737166) and Md Mubashiruddin (160923737167)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in **Information Technology** is a record of Bonafide work carried out by them.

Internal Project Guide
Ms. M Shilpa
Assistant Professor

Dr. K. Nagi Reddy
H.O.D, I.T

External Examiner



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY
(UGC Autonomous)
(Affiliated to Osmania University)
Himayathsagar, Hyderabad, 500091
Department of Information Technology

DECLARATION BY THE CANDIDATES

We, **Mohammed Sarwar Khan (160923737142), Syed Mohd Hifzan (160923737166) and Md Mubashiruddin (160923737167)**, hereby declare that the Project Phase-I report entitled **ReconIQ: An Agentic Financial Reconciliation System** is carried out under the guidance of **Ms. M Shilpa** Assistant Professor, Department of Information Technology, **Lords Institute of Engineering and Technology**, **Hyderabad** is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in **Information Technology.**

This is a record of Bonafide work carried out by us and the results embodied in this project have not been reproduced or copied from any source. The results embodied in this project report have not been submitted to any other university or institute for the award of any other degree or diploma.

Mohammed Sarwar Khan (160923737142)
Syed Mohd Hifzan (160923737166)
Md Mubashiruddin (160923737167)

ACKNOWLEDGMENT

We are very pleased to present this thesis of our project. This period of our student life has been truly rewarding a number of people were of immense help to us during the course of our research and the preparation of our thesis.

First, we wish to thank **GOD Almighty** who created heavens and earth, who helped us in completing this project and we also thank our parents who encouraged us in this period of research.

We would like to express our sincere gratitude to **The Management** of **Lords Institute of Engineering and Technology, Hyderabad** for providing us with an excellent platform and resources to successfully carry out this project.

We would also like to thank **Dr. Ravi Kishore Singh, Principal, L.I.E.T.** for extending his help in overall completion of this project work.

We would like to express my deep sense of gratitude **to Dr. K. Nagi Reddy, Professor & HOD**, **Information Technology, Lords Institute of Engineering & Technology, Hyderabad,** for his encouragement and cooperation throughout the project.

We would like to thank **Ms. M Shilpa, Assistant Professor, Dept. of Information Technology, Lords Institute of Engineering & Technology**, Hyderabad, our project internal guide, for her guidance and help. Her insight during the course of our research and regular guidance were invaluable to us.

Lastly, we acknowledge the support of our faculty members, friends, and family, who inspired and encouraged us throughout this journey.

Mohammed Sarwar Khan (160923737142)
Syed Mohd Hifzan (160923737166)
Md Mubashiruddin (160923737167)

Vision of the Institute

Lords Institute of Engineering and Technology strives for excellence in professional education through quality, innovation and teamwork and aims to emerge as a premier institute in the state and across the nation.

Mission of the Institute

- To impart quality professional education that meets the needs of present and emerging technological world.
- To strive for student achievement and success, preparing them for life, career and leadership.
- To provide a scholarly and vibrant learning environment that enables faculty, staff and students to achieve personal and professional growth.
- To contribute to advancement of knowledge, in both fundamental and applied areas of engineering and technology.
- To forge mutually beneficial relationships with government organizations, industries, society and the alumni.

PRINCIPAL
Principal
Lords Institute of Engineering & Tech.
(An Autonomous Institution)
Sy. No. 32, Himayath Sagar, Hyderabad-500091.T.S.

LORDS INSTITUTE OF ENGINEERING &TECHNOLOGY

Approved by AICTE/Affiliated to Osmania University/Estd.2003.
Accredited 'A' grade by NAAC
(UGC Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

Vision of the Department:

To be a frontier in Information Technology focusing on quality professional education, innovation and employability.

Mission of the Department:

DM1: Adopt innovative teaching learning processes to achieve stated outcomes and sustained performance levels.

DM2: Provide learner centric environment to enrich knowledge, skills and innovation through industry academia collaborations.

DM3: Train the concerned stakeholders on latest technologies to meet the industry needs.

DM4: Inculcate leadership, professional, interpersonal skills and ethical values to address the contemporary societal issues.

Note: DM: Department Mission

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technolo
Himayath Sagar, Hyderabad-91

B.E Information Technology Program Educational Objectives (PEOs):

PEO1	Be a competent software engineer in IT and allied industry providing viable solutions.
PEO2	Exhibit professionalism, engage in lifelong learning and adopt to changing industry and societal needs.
PEO3	Be effective as an individual and also cohesive in a group to execute multidisciplinary projects.
PEO4	Pursue higher studies in Information Technology with emphasis on competency and innovation.

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technolo
Himayath Sagar, Hyderabad-91

B.E Information Technology Program Outcomes (POs):

Engineering Graduates will be able to:

S. No.	Program Outcomes (POs)
1.	**PO1: Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
2.	**PO2: Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
3.	**PO3: Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
4.	**PO4: Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
5.	**PO5: Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
6.	**PO6: The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
7.	**PO7: Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
8.	**PO8: Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
9.	**PO9: Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
10.	**PO10: Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
11.	**PO11: Life-Long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-91

B.E Information Technology Program Specific Outcomes (PSOs):

PSO1	**Professional Skills:** Identify, analyze requirements, design and implement computer programs in the area of artificial intelligence, machine learning, big data analytics and networking of varying complexity.
PSO2	**Problem-Solving Skills:** Deliver quality service to IT industry and provide solutions to real-life problems applying modern computer languages, software methods and tools.

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-91

Course Outcomes: Project Phase- I

After completion of the project, the student will be able to

S.No	Description	Blooms Taxonomy Level
PP.I.CO1	Demonstrate the ability to synthesize and apply the knowledge and skills acquired in the academic program to the real-world problems.	BTL3
PP.I.CO2	Evaluate different solutions based on economic and technical feasibility	BTL4
PP.I.CO3	Effectively plan a project and confidently perform all aspects of project management.	BTL5
PP.I.CO4	Demonstrate effective written and oral communication skills	BTL4
PP.I.CO5	Prepare the documentation report and perform the presentation of the project work	BTL3

LORDS INSTITUTE OF ENGINEERING &TECHNOLOGY
Approved by AICTE/Affiliated to Osmania University/Estd.2002.
Accredited 'A' grade by NAAC
(UGC Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

Course Articulation Matrix:

Mapping of Course Outcomes (COs) with Program Outcomes (POs) and Program Specific Outcomes (PSOs):

Course Outcomes (COs)	Program Outcomes (POs)											Program Specific Outcomes (PSOs)	
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO 10	PO 11	PSO1	PSO2
PP.I.CO1	3	2	2	-	2	2	-	-	3	-	-	2	2
PP.I.CO2	3	3	2	3	2	-	2	-	3	-	-	-	2
PP.I.CO3	3	3	3	-	3	3	1	2	2	3	2	3	-
PP.I.CO4	3	2	2	3	3	3	3	2	-	-	2	3	3
PP.I.CO5	3	3	3	3	3	3	3	2	3	3	-	3	3
AVG	**3**	**2.6**	**2.5**	**3**	**2.6**	**2.7**	**2.2**	**2**	**2.9**	**3**	**2**	**2.7**	**2.5**

Level:

1- Low correlation (Low), 2- Medium correlation (Medium), 3-High correlation (High)

SDG Mapping:

SDG	Mapped Indicator	SDG	Mapped Indicator	SDG	Mapped Indicator
1 NO POVERTY		7 AFFORDABLE AND CLEAN ENERGY		13 CLIMATE ACTION	
2 ZERO HUNGER		8 DECENT WORK AND ECONOMIC GROWTH		14 LIFE BELOW WATER	
3 GOOD HEALTH AND WELL-BEING	✓	9 INDUSTRY, INNOVATION AND INFRASTRUCTURE		15 LIFE ON LAND	
4 QUALITY EDUCATION		10 REDUCED INEQUALITIES		16 PEACE, JUSTICE AND STRONG INSTITUTIONS	
5 GENDER EQUALITY		11 SUSTAINABLE CITIES AND COMMUNITIES		17 PARTNERSHIPS FOR THE GOALS	
6 CLEAN WATER AND SANITATION		12 RESPONSIBLE CONSUMPTION AND PRODUCTION			

ABSTRACT

Financial reconciliation is a critical activity in organizations processing payments across multiple systems. Transactions may appear differently in bank statements, internal ledgers, and payment gateway exports. Settlement batching, gateway fees, inconsistent descriptions, date variations, and similar transactions make manual matching slow and error-prone. This project introduces ReconIQ, an agentic financial reconciliation system that matches transactions, explains decisions, and identifies unresolved cases.

ReconIQ uses a deterministic fast path for straightforward matches. It first checks transaction reference numbers and amount consistency. When reliable references are unavailable, it applies configurable tolerances, including approximately ±1% amount variation and ±3 days date variation. A single qualifying candidate is confirmed automatically. Multiple candidates are classified as ambiguous, while unmatched records remain unresolved. Both cases enter an agent workflow for further investigation.

The agent workflow follows a proposer-verifier architecture. The Proposer investigates unresolved transactions using available evidence and tools. The Verifier independently evaluates proposed matches, detecting decoy records, insufficient evidence, amount or date inconsistencies, and incorrect settlement combinations. Unsupported decisions remain exceptions for manual review. Supporting tools include sum_check for many-to-one settlement analysis and description_similarity for comparing transaction descriptions.

The backend uses Python, FastAPI, and asynchronous job execution. It accepts CSV inputs, stores results in JSON files, and provides APIs for initiating reconciliation, tracking execution, retrieving results, inspecting reasoning traces, and accessing exceptions. The frontend uses Next.js, React, TypeScript, TanStack Query, and Tailwind CSS. Agent inference supports local execution through Ollama with Qwen2.5:7B, alongside Groq, Gemini, and automatic provider routing.

Keywords: Financial Reconciliation, Payment-Gateway Exports, Deterministic Matching, Agentic AI, Proposer-Verifier, FastAPI, Next.js, Asynchronous Processing, Audit Trail.

CONTENTS

Certificate i

Declaration ii

Acknowledgement iii

Vision and Mission of the Institute iv

Vision and Mission of the Department v

Program Educational Objectives vi

Program Outcomes vii

Program Specific Outcomes viii

Course Outcomes ix

Course Articulation Matrix x

Abstract xi

Contents xii

List of Figures xv

List of Tables xvi

CHAPTER-1: INTRODUCTION 1

1.1 Introduction

1.2 Problem Statement

1.3 Objectives

1.4 Scope of the Project

1.5 Existing System

1.6 Proposed System

1.7 Advantages

CHAPTER-2: LITERATURE SURVEY 6

2.1 Introduction

2.2 Financial Reconciliation Systems

2.3 Deterministic Record Matching

2.4 Agentic AI and Proposer-Verifier Design

2.5 Frontend Technologies

2.6 Data Handling and Persistence

2.7 API and Asynchronous Processing

2.8 LLM Providers and Tool Calling

2.9 Security and Auditability

2.10 Comparative Analysis

2.11 Summary

CHAPTER-3: SYSTEM ANALYSIS AND REQUIREMENTS 12

3.1 Introduction

3.2 Functional Requirements

3.3 Non-Functional Requirements

3.4 Hardware Requirements

3.5 Software Requirements

3.6 User Roles

3.7 Feasibility Study

3.8 Use Case Overview

3.9 Summary

CHAPTER-4: SYSTEM DESIGN AND ARCHITECTURE 19

4.1 Introduction

4.2 System Architecture

4.3 Major System Modules

4.4 Data Persistence Design

4.5 REST API Design

4.6 Runtime Architecture

4.7 Summary

CHAPTER-5: IMPLEMENTATION TESTING 26

5.1 Introduction

5.2 Development Environment Setup

5.3 Backend Implementation

5.4 Front Implementation

5.5 File-Based Data Persistence

5.6 Agent Tools Implementation

5.7 Asynchronous Processing and Provider Routing

5.8 Testing Strategy

5.9 Sample Test Cases

5.10 Performance and Deployment Evaluation

5.11 Summary

CHAPTER-6: RESULTS AND FUTURE ENHANCEMENTS 37

6.1 Introduction

6.2 Expected Results

6.3 Result Evaluation

6.4 Project Contributions

6.5 Limitations

6.6 Future Enhancements

CONCLUSION 42

REFERENCES 43

List of Figures

SI.no	Fig.no	Name of the Figure	Page No
1	4.2	ReconIQ System Architecture	20
2	4.6	ReconIQ Deployment Architecture	25
3	5.3	Backend Architecture	29
4	5.4	Frontend Architecture	30
5	5.5	Data Persistence Architecture	31
6	5.6	Agent Tools Workflow	32
7	5.7	Asynchronous Processing and Provider Routing	33

List of Tables

SI.no	Table.no	Name of the Table	Page No
1	2.10	Comparative Analysis	10
2	3.4	Hardware Requirements	15
3	3.5	Software Requirements	15
4	4.4	Database Design	23
5	4.5	REST API Design	24
6	5.9	Sample Test Cases	34
7	6.3	Result Evaluation	38

CHAPTER-1
INTRODUCTION

1.1 Introduction

ReconIQ is an agentic financial reconciliation system intended to compare transaction records drawn from bank statements, internal ledgers, and payment-gateway exports. Financial systems often describe the same economic event using different references, descriptions, timestamps, or settlement structures. These differences complicate reconciliation and make simple one-to-one comparison insufficient for some transactions.

The application uses a staged approach. A deterministic matching layer handles cases that can be resolved using reference numbers, amount consistency, and date or amount tolerances. It sends ambiguous and unresolved records to an agent workflow, where a Proposer examines available evidence and a Verifier independently challenges the proposed match. When the available evidence remains insufficient, the system can preserve the record as an exception rather than force a decision.

The backend uses Python and FastAPI, with asynchronous job processing to avoid holding a single HTTP request open throughout a lengthy reconciliation run [2], [9]. Uploaded source data is supplied as CSV. Completed results are persisted as JSON files. The frontend uses Next.js, React, TypeScript, TanStack Query, and Tailwind CSS to initiate runs, poll execution status, and display results supplied by the backend.

The project supports payment-gateway exports as one of its reconciliation inputs. The supplied implementation notes do not establish whether ReconIQ creates Razorpay orders, verifies payment signatures, receives Razorpay webhooks, or requests live payment data through Razorpay APIs. Those functions are outside the confirmed implementation description unless the project team supplies further evidence.

1.2 Problem Statement

Organizations may need to reconcile transactions across systems whose records differ in format and level of detail. A bank settlement may represent several internal ledger entries after fees. A payment gateway and a bank may use different transaction descriptions. Several records may have similar amounts and dates, and some apparent candidates may be decoys. A

genuine anomaly may have no defensible match at all.

Basic exact-match scripts cannot reliably handle all these cases. Broad fuzzy matching can incorrectly pair records that happen to be numerically or textually similar. Unverified AI-generated matches create another risk because a language model can propose a plausible but incorrect explanation. Synchronous processing can also make the application difficult to monitor when an agent requires multiple rounds of investigation.

The problem addressed by ReconIQ is to design a reconciliation workflow that combines clear deterministic rules, controlled AI-assisted investigation for ambiguous cases, independent verification, asynchronous run management, persistent results, and an explicit list of unresolved exceptions.

1.3 Objectives

The primary objectives of the project are:

1. To reconcile transaction records from bank statements, internal ledgers, and payment-gateway exports.
2. To match records by reference number when reliable references are available.
3. To verify amount consistency before confirming reference-based matches.
4. To apply configurable amount and date tolerances when a direct reference match is not available.
5. To distinguish confirmed, ambiguous, unresolved, and exception cases.
6. To route ambiguous records through a Proposer-Verifier workflow.
7. To support many-to-one settlement analysis using a sum-check tool.
8. To use description similarity as supporting evidence without treating low text similarity as an automatic rejection.
9. To execute reconciliation jobs asynchronously and expose runtime status through an API.
10. To persist completed output in JSON and provide access to results, exception lists, and reasoning traces.
11. To evaluate matching quality using held-out ground truth and measures such as precision, recall, match rate, and processing speed.
12. To document verified implementation results, limitations, and future work without presenting unexecuted tests as completed.

1.4 Scope of the Project

The confirmed scope covers the processing and reconciliation of transaction data supplied through CSV files, deterministic reference and tolerance matching, AI-assisted resolution of difficult cases, verification of agent proposals, asynchronous execution, frontend status reporting, and persistent result retrieval.

The system's documented inputs and outputs include:

- Bank statement records.
- Internal ledger records.
- Payment-gateway export records.
- Reference-based and tolerance-based match candidates.
- Many-to-one settlement candidate combinations.
- Reconciliation run status and progress.
- Completed result files in JSON format.
- Exception lists for records that cannot be safely resolved.
- Reasoning-trace retrieval for selected records.
- Evaluation against a ground-truth mapping that is kept separate from the matching pipeline.

1.5 Existing System

Traditional reconciliation may rely on manual spreadsheet comparisons, strict database joins, or scripts based on exact amounts and transaction identifiers. Such techniques are useful where source data is standardized, but their reliability declines when identifiers are missing, settlement records aggregate several entries, or descriptions differ substantially.

Common limitations include:

- Manual effort when records use inconsistent descriptions or references.
- Failure to identify a settlement that combines multiple ledger entries.
- Incorrect matches where several candidates have similar amounts and dates.
- False positives from indiscriminate fuzzy matching.
- Difficulty documenting why a record was paired with a particular candidate.

- Lack of an explicit exception workflow for transactions that cannot be reconciled safely.
- Long-running reconciliation work that blocks API responses or provides limited progress information.
- Risk of evaluation leakage if labels used to generate synthetic data are visible to the matcher.

1.6 Proposed System

ReconIQ separates reconciliation into deterministic and agentic stages. The deterministic fast path first attempts reference-based matching and checks amount consistency. For cases without a reliable reference match, it applies approximate tolerances of plus or minus one percent for amount and plus or minus three days for date, as described in the supplied project notes. One qualifying candidate can be confirmed. Multiple candidates are considered ambiguous, and zero candidates remain unresolved.

The agent layer investigates cases that need additional judgment. The Proposer selects and explains a candidate from the available evidence. The Verifier independently looks for incorrect candidates, decoys, weak evidence, inconsistent amounts or dates, and invalid settlement combinations. A disagreement can trigger a bounded retry with the objection included. If the case remains unclear, the workflow records an exception.

Specialist tools support this workflow. sum_check evaluates many-to-one settlement combinations against the target amount and configured fee tolerance. The candidate ranking logic prioritizes settlement consistency before numerical closeness because the supplied notes document a case where an incorrect cross-batch combination was numerically closer than the correct one. description_similarity provides additional semantic evidence without acting as a strict rejection rule.

The backend runs reconciliation as an asynchronous job, and the frontend polls status from the backend. This makes progress observable while longer agent-based decisions execute. Completed results are saved as JSON files and made available through result and exception endpoints.

1.7 Advantages

The proposed design has the following advantages:

- Straightforward records can be handled without an LLM call.
- Reference matches receive an amount sanity check.
- Tolerance-based matching can account for limited date and amount variation.
- Ambiguous records are not automatically treated as confirmed matches.
- The Verifier provides a separate review of the Proposer's evidence.
- Many-to-one settlement combinations can be evaluated with sum-based checks.
- Description similarity supplements rather than overrides financial evidence.
- Unresolved cases remain visible through an exception workflow.
- Asynchronous jobs let the frontend report runtime status.
- Persisted results can be retrieved after the run has completed.
- Ground-truth separation supports more credible evaluation of matching quality.

CHAPTER-2
LITERATURE SURVEY

2.1 Introduction

This chapter reviews the concepts and technologies relevant to ReconIQ. These include financial reconciliation, deterministic record matching, agentic AI, independent verification, web application architecture, asynchronous APIs, structured file persistence, tool-calling language models, and auditable decision workflows. The technical choices described below are drawn from the supplied project notes and official documentation where applicable.

2.2 Financial Reconciliation Systems

Financial reconciliation compares representations of financial events held by separate records or systems. Typical source data can include bank statement lines, internal accounting entries, and payment-gateway settlement exports. A reconciliation system needs to determine whether the records represent the same event while accounting for legitimate differences in representation.

One-to-one matching is only one pattern. A payment gateway may collect several transactions and transfer a consolidated settlement after fees. This creates a many-to-one relationship between ledger records and the bank settlement. Descriptions can differ even when the amount and underlying event correspond. A robust system must therefore preserve transaction evidence, represent relationships clearly, and support exception handling. Where the application eventually calls Razorpay directly, the integration contract should follow the official API documentation [1] and be tested in a controlled environment.

ReconIQ models reconciliation as a decision process rather than a single fuzzy-score calculation. It distinguishes direct matches, tolerance matches, ambiguous candidate sets, and unresolved items. This allows the application to make a conservative decision when the evidence does not support a unique match.

2.3 Deterministic Record Matching

Deterministic matching uses explicit rules and repeatable calculations. The supplied implementation first looks for reference-number matches where available. It then checks amount consistency so that a shared reference does not automatically override a suspicious financial mismatch.

For records without a reliable reference match, the documented fast path considers an approximate amount difference of plus or minus one percent and a date difference of plus or minus three days. The tolerance stage requires a unique candidate for confirmation. More than one candidate produces an ambiguous result, and no candidates produce an unresolved result.

These limits are configuration-dependent and should be validated against the intended transaction sources. A tolerance can help handle settlement timing and small differences, but a broad tolerance can also create false matches. For this reason, ReconIQ sends ambiguous cases to the agent workflow rather than choosing the candidate with the nearest amount or date alone.

2.4 Agentic AI and Proposer-Verifier Design

A language model can help interpret inconsistent descriptions, inspect candidate records, and assemble evidence for cases that are difficult to resolve with direct rules. However, model output is probabilistic and may be incorrect even when the explanation sounds reasonable. The project therefore assigns separate roles to a Proposer and a Verifier.

The Proposer investigates one ambiguous or unresolved record and suggests a match using the available tools and record evidence. The Verifier independently challenges that proposal. It looks for decoys, weak evidence, amount or date inconsistencies, incorrect candidate selection, and invalid settlement combinations. If the Verifier disagrees, the Proposer can retry with the objection. When the available evidence remains insufficient, the workflow records an exception instead of forcing a match.

This architecture is intended to reduce the risk of an unchecked agent decision. It does not guarantee correctness. The value of the design must be assessed through measured precision, recall, false-positive analysis, latency, and review of failure cases.

2.5 Frontend Technologies

The project frontend uses Next.js, React, TypeScript, TanStack Query, and Tailwind CSS, as specified in the project notes. React supports component-based user interfaces, while Next.js provides an application framework for organizing the frontend. TypeScript adds static type checking to the JavaScript development workflow. TanStack Query manages server state and request lifecycle behavior, and Tailwind CSS supplies utility classes for interface styling [8]. Official documentation for these tools is listed in the References section [3]-[6], [8].

The frontend is driven by backend API data rather than hardcoded progress values. It submits or accesses reconciliation runs, polls backend status, and presents the persisted results. The supplied notes specifically mention a Runs control room that uses /api/results/latest to retrieve the newest completed result. Detailed screen designs, access-control behavior, and user-interface screenshots should be added only where they correspond to the current implementation.

2.6 Data Handling and Persistence

The supplied project uses CSV as its input format and JSON files to persist completed results. The project directory also includes locations for uploaded files, logs, run state, and result artifacts. The notes do not identify a relational database as part of the current architecture.

CSV is a practical format for reconciliation inputs because transaction data can be exported from many financial systems into rows and columns. However, source files can differ in column names, date formats, amount representation, currency conventions, and identifier fields. The import process should validate required fields, normalize values where safe, preserve the original source information, and report malformed records clearly.

JSON is used to store completed result objects. The storage layer must distinguish the newest completed run from older files, which is important because the supplied notes document a stale-latest-result bug that was corrected by selecting persisted results newest-first. Durable result storage should be tested for partial writes, concurrent runs, malformed files, and recovery after process restart.

2.7 API and Asynchronous Processing

FastAPI is the Python API framework specified in the backend implementation. Its official documentation describes a framework for building APIs with Python type hints [2]. The project exposes endpoints to start reconciliation jobs, query their status, retrieve run results, obtain the latest completed result, inspect record-level reasoning traces, and retrieve exceptions.

Agent-based resolution can require multiple model calls and tool interactions. Waiting for a complete batch within one synchronous HTTP request would delay the response and make the application harder to monitor. ReconIQ therefore executes a run as an asynchronous job. The client polls status endpoints and uses real backend state to display progress. The implementation also supports bounded concurrency for processing multiple agent records.

Asynchronous processing introduces its own requirements. The backend must maintain a stable run identifier, distinguish running, completed, and failed states, handle partial failures, and avoid reporting completion before durable results are written. It must also prevent duplicate job execution where a request is retried, if exactly-once or idempotent behavior is a requirement of the final design.

2.8 LLM Providers and Tool Calling

The project notes identify four provider modes: local, Groq, gemini, and auto. Local inference uses Ollama with the qwen2.5:7b model. The notes state that cloud providers use OpenAI-compatible tool-calling interfaces so the same agent logic can operate across providers. Actual provider versions, model identifiers for remote services, rate limits, and API configuration should be recorded.

The agent workflow uses tools to obtain structured evidence. sum_check searches combinations of candidate ledger records to explain a target settlement within configured fee tolerance. The implementation ranks candidates by settlement consistency first and numerical closeness second. description_similarity measures semantic support when descriptions differ, but low similarity does not automatically reject a candidate.

Tool calling allows a model to request execution of defined functions and use their results in a subsequent response. Ollama's published documentation describes tool calling through the API's tool definitions [7]. This capability does not remove the need for validation: tool

arguments, returned candidate identifiers, amounts, and final decisions should be checked by deterministic code before a match is persisted.

2.9 Security and Auditability

Financial reconciliation data can contain personally identifiable information, account details, merchant references, and other confidential records. The report does not establish which authentication, authorization, encryption, or retention controls are implemented. These details must be confirmed before the system is described as production-ready.

The design should limit access to uploaded files and results, avoid exposing secrets in logs, keep provider API keys outside source control, and minimize the financial data sent to external model providers. Where cloud models are used, the project should document which fields are transmitted, how credentials are stored, and whether data retention or regional restrictions apply.

Auditability requires retaining enough evidence to explain why a record was matched or left unresolved. Useful fields include the source record identifier, candidate identifiers, amount and date comparisons, tool outputs, Proposer decision, Verifier objection or agreement, retry count, terminal decision, and exception reason. A reasoning trace should be treated as supporting evidence, not proof that the selected candidate is correct.

2.10 Comparative Analysis

Technology or Approach	Purpose	Benefit
Reference-based matching	Match using transaction references	Fast and deterministic where references are reliable
Tolerance matching	Identify candidates by amount and date	Handles limited source variation
Proposer-Verifier agents	Investigate and challenge ambiguous matches	Adds a separate review step
`sum_check`	Evaluate many-to-one settlements	Detects plausible settlement combinations
`description_similarity`	Compare differing descriptions	Supplies semantic supporting evidence

FastAPI	Expose backend endpoints	Structured API interface for the web client
Next.js and React	Frontend application	Component-based interface with backend-driven state
TanStack Query	Manage server-state requests	Supports polling and request lifecycle management
Ollama with `qwen2.5:7b`	Local model inference	Enables local execution in a supported environment

2.11 Summary

The literature and technology review supports a hybrid architecture for financial reconciliation. Deterministic rules address cases with clear evidence, while agent-based investigation is reserved for ambiguity. Independent verification and exception handling reduce the pressure to accept uncertain matches. FastAPI, asynchronous jobs, CSV inputs, JSON persistence, and a backend-driven frontend provide the application infrastructure. Quality must be established through evaluation against separated ground truth and explicit reporting of failure cases.

CHAPTER-3
SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Introduction

System analysis defines ReconIQ's expected behavior, technical constraints, operating roles, and evaluation approach. The requirements below are based on the supplied project notes. Features that the notes do not verify, such as direct Razorpay API/webhook calls, authentication screens, container orchestration, and production deployment, are listed as confirmation items rather than current capabilities.

3.2 Functional Requirements

FR1: Input File Processing
The system shall accept reconciliation inputs in the supported CSV format and identify malformed or incomplete records.

FR2: Reference Matching
The system shall attempt to match records using reference numbers when reliable references are available.

FR3: Amount Consistency Check
The system shall check the amount associated with a reference match and flag suspicious inconsistencies instead of confirming the match automatically.

FR4: Tolerance-Based Matching
Where a reliable reference match is unavailable, the system shall identify candidates using configured amount and date tolerances. The supplied configuration is approximately plus or minus one percent for amount and plus or minus three days for date.

FR5: Candidate Classification
The system shall confirm only a uniquely qualified candidate through the fast path. Multiple candidates shall be classified as ambiguous, and zero candidates shall be classified as unresolved.

FR6: Agent-Based Investigation

The system shall route ambiguous or unresolved records to the agent workflow when that workflow is enabled.

FR7: Independent Verification

The system shall provide a Verifier role that reviews the Proposer's suggested match and can agree, object, or cause a bounded retry.

FR8: Many-to-One Settlement Checks

The system shall support candidate-combination analysis through sum_check, including comparison against a target settlement amount and configured fee tolerance.

FR9: Description Similarity

The system shall provide description similarity as supporting evidence without treating low similarity as an automatic rejection.

FR10: Asynchronous Run Management

The system shall start reconciliation as a job, expose run status, and allow a client to poll for progress.

FR11: Result Persistence and Retrieval

The system shall save completed results as JSON files and expose endpoints for retrieving a run's result and the latest completed result.

FR12: Exception and Reasoning Trace Access

The system shall expose exception lists and record-level reasoning traces for investigation and audit purposes.

FR13: Evaluation Against Ground Truth

The evaluation process shall compare the output with a ground-truth mapping that is isolated from the matching pipeline.

FR14: Provider Selection

The agent layer shall support the documented provider modes: local, Groq, Gemini, and automatic routing, subject to the configured provider credentials and model availability.

3.3 Non-Functional Requirements

Correctness: Confirmed matches should be supported by reproducible evidence. Ambiguous and unsupported cases should not be converted into matches solely to increase coverage.

Reliability: The system should handle malformed input, empty model responses, API/tool-call failures, and persistence errors without recording an unsupported decision as confirmed.

Performance: Deterministic matching should avoid unnecessary model calls. Long-running agent operations should not block status polling. Latency and throughput must be measured under a documented workload.

Scalability: Agent records should be processed with bounded concurrency. Increasing concurrency must not exhaust provider limits, memory, or local compute resources.

Auditability: The application should retain sufficient evidence to identify the candidate, supporting checks, agent decisions, objections, retries, terminal result, and exception reason.

Maintainability: Matching logic, agent tools, API routes, evaluation routines, and frontend views should remain modular.

Usability: The frontend should show accurate job state, completion status, result availability, and unresolved exceptions using backend data.

Portability: The backend and frontend should be capable of running in supported development environments after their documented dependencies are installed.

Security and Privacy: Secrets should not be committed to the repository. Access to financial input and result files should be controlled. The project should document how data is handled when a cloud LLM provider is selected.

Recoverability: Persisted results should remain retrievable after completion and the latest-results endpoint should reliably select the newest completed run.

3.4 Hardware Requirements

The following are suggested development requirements, not mandatory production specifications.

Components	Requirement or Configuration
Processor	Intel I7 14th Gen
Memory	32 Gb
Storage	512 Gb
GPU	RTX 5070 12 Gb vram
Network	Internet connection required for dependency downloads and cloud model
Local inference environment	Ollama with qwen2.5:32b; machine requirements used in testing

3.5 Software Requirements

Software or Component	Purpose
Python	Backend language and reconciliation logic
FastAPI	Backend API framework
Uvicorn	Local ASGI server used by documented startup command
CSV data	Reconciliation input
JSON files	Persistent results
Next.js	Frontend application framework
React	Frontend component library
TypeScript	Frontend type system
TanStack Query	Backend request and status-state management

3.6 User Roles

The following roles describe the operational responsibilities around the system. They are not a claim that the current version implements authenticated user accounts or role-based access control.

Reconciliation Operator: Selects or uploads supported input files, starts a run, and monitors its status.

Financial Reviewer: Reviews confirmed matches, ambiguous records, reasoning traces, and exceptions. The API exposes reasoning traces and exception data for review; a separate authenticated approval-and-override workflow is not part of the documented implementation.

System Administrator: Maintains environment configuration, provider credentials, application settings, and access to execution logs, where such administrative controls exist.

3.7 Feasibility Study

Technical Feasibility: The project uses established web technologies and model interfaces. The documented deterministic matcher, tools, APIs, asynchronous job workflow, and frontend stack form a feasible implementation approach. Actual deployment scale depends on data volume, provider limits, compute resources, and persistence design.

Economic Feasibility: Deterministic matching reduces the number of cases sent to language models. Local inference can reduce external API usage but requires suitable computing resources. Cloud provider use may introduce usage-based costs. No formal project-cost measurement is reported.

Operational Feasibility: CSV input, run-status polling, persisted results, and exception retrieval provide a practical workflow for operators. Operational readiness still depends on validated input schemas, data-access controls, error recovery, and reviewer procedures.

Schedule Feasibility: The modular structure allows implementation and testing in stages: input processing, deterministic matching, agent tools, proposer-verifier orchestration, asynchronous APIs, frontend integration, and benchmark evaluation. Development proceeded through implementation of data generation, matching logic, agent tools, API processing, frontend integration, and evaluation.

Integration Feasibility: The confirmed integration path is file-based. Razorpay transaction or settlement exports can be used when their fields are compatible with, or mapped to, the application's supported CSV schema. Live Razorpay API and webhook connectivity are outside the current scope.

3.8 Use Case Overview

The principal use cases are:

1. Prepare bank statement, ledger, and payment-gateway input files.
2. Validate and load CSV input records.
3. Run deterministic reference matching.
4. Apply amount and date tolerance checks when needed.
5. Route ambiguous or unresolved records to the agent workflow.

6. Investigate candidate combinations for many-to-one settlements.
7. Verify the Proposer's decision independently.
8. Mark unresolved records as exceptions.
9. Poll the status of an asynchronous run.
10. Retrieve completed results from persistent storage.
11. Retrieve the latest completed result.
12. Inspect a record-level reasoning trace.
13. Review exception records.
14. Compare results with held-out ground truth during evaluation.
15. Import Razorpay transaction or settlement export data through the supported CSV input workflow.

3.9 Summary

The requirements establish a conservative and auditable reconciliation process. Reference and tolerance rules handle straightforward cases, while ambiguous cases can be escalated to an agent workflow with independent verification. Asynchronous execution, durable results, APIs for status and exceptions, and evaluation against isolated ground truth support operational visibility and quality measurement. The current Razorpay integration is file-based. Authentication, role-based authorization, and production infrastructure are outside the current documented scope.

CHAPTER-4
SYSTEM DESIGN AND ARCHITECTURE

4.1 Introduction

This chapter describes ReconIQ's logical layers, major modules, data and persistence design, API structure, and deployment approach. The architecture is based on the supplied project details: a Python/FastAPI backend, a Next.js frontend, CSV input, JSON result persistence, asynchronous reconciliation jobs, and a provider-aware agent workflow.

4.2 System Architecture

ReconIQ can be represented as five connected layers.

Input Layer: Accepts bank statement, internal ledger, and payment-gateway CSV exports. Input parsing and normalization identify records and the fields available for matching.

Deterministic Matching Layer: Uses reference numbers when available, checks amount consistency, and applies amount and date tolerances for candidate discovery. It confirms a candidate only when the deterministic rules identify one defensible match.

Agentic Resolution Layer: Processes ambiguous or unresolved cases. The Proposer investigates a case and invokes tools where needed. The Verifier challenges the candidate and evidence. An unresolved disagreement is retained as an exception.

Application and Persistence Layer: FastAPI endpoints initiate runs, expose status, return results and exceptions, and retrieve reasoning traces. Asynchronous job state and JSON result files support the long-running workflow and result access.

Presentation Layer: The Next.js frontend uses React, TypeScript, TanStack Query, and Tailwind CSS. It reads status and result data from the backend and displays runtime progress based on actual API responses.

The project also includes evaluation logic that compares reconciliation output with a ground-truth mapping kept separate from the matching pipeline. The diagram below should be generated only as a visual representation of these documented components.

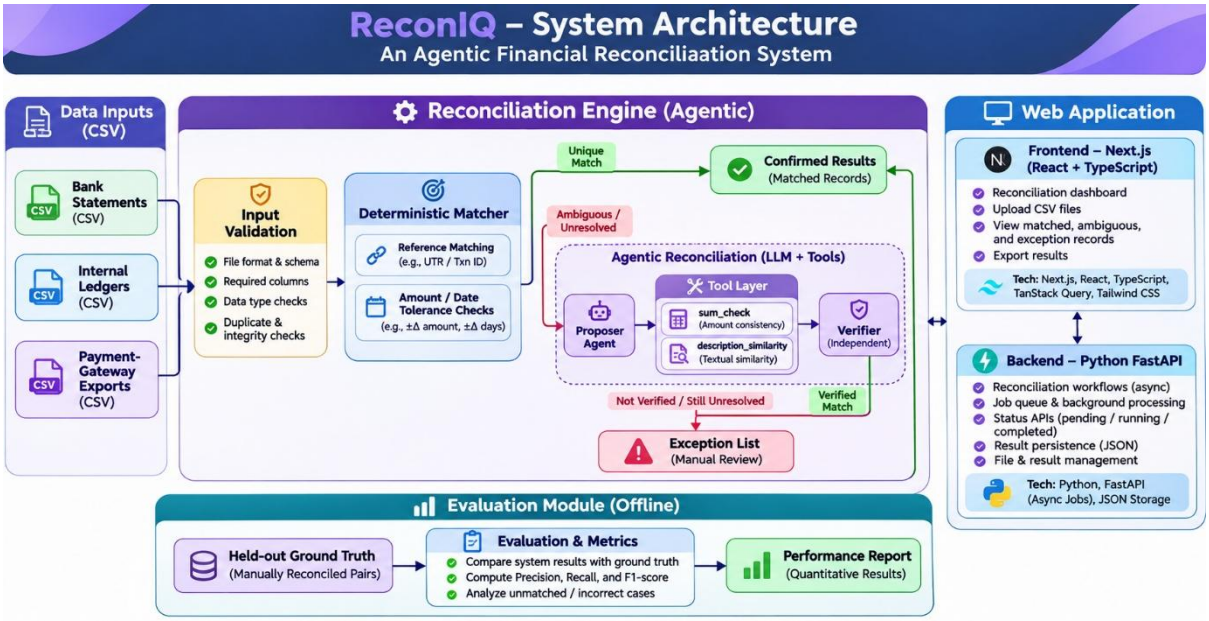


Figure 4.2: ReconIQ System Architecture

4.3 Major System Modules

4.3.1 Input Processing Module

This module accepts CSV-based records from the supported sources. It must identify source-specific columns and preserve record identity while preparing data for matching. Fields such as reference number, amount, transaction date, description, and settlement grouping may be available depending on the export schema. Input records are normalized to the supported CSV schema before matching.

Input validation should distinguish malformed data from valid records that simply lack a reference. Missing values must not be converted into fabricated identifiers. The system should report unsupported formats and data-quality errors before they contaminate evaluation.

4.3.2 Deterministic Matching Module

The fast path first attempts reference-based matching. When a reference candidate exists, the system checks amount consistency. A suspicious mismatch is flagged rather than confirmed. If no reliable reference candidate is available, the matcher applies the documented approximate amount and date tolerances.

4.3.3 Agent Orchestration Module

The agent orchestration module receives records that the deterministic matcher cannot safely resolve. It provides the Proposer with relevant candidate records and available tools, tracks model interactions within bounded limits, and passes the proposed decision to the Verifier. The module must reject empty or structurally invalid terminal decisions.

4.3.4 Proposer Module

The Proposer examines one unresolved or ambiguous record and suggests a candidate match using the available evidence. Its output should identify the selected record or records and explain the financial evidence supporting the proposal. A language-model explanation alone is not enough to establish correctness; the output must be validated against the candidate set and record data.

4.3.5 Verifier Module

The Verifier independently challenges the Proposer's output. It looks for incorrect candidates, planted decoys, weak evidence, amount/date mismatches, and incorrect settlement combinations. Where it disagrees, the objection can be passed into a bounded retry. If the case remains unclear after the allowed attempts, the system should leave it unresolved or record an exception.

4.3.6 Agent Tools Module

sum_check: Searches candidate combinations whose total can explain the target settlement within configured fee tolerance. The supplied notes record a bug in which an incorrect cross-batch combination was numerically closer than the correct settlement. The ranking logic was changed to prioritize settlement consistency before numerical closeness.

description_similarity: Supplies semantic similarity evidence for records whose descriptions differ. Low similarity does not automatically reject a match because the same transaction can be described differently across systems.

Tool outputs should be deterministic where possible and should include the candidate identifiers and calculations needed for verification

4.3.7 Asynchronous Run Management Module

The backend treats reconciliation as an asynchronous job. The client receives a run identifier and polls the status endpoint. Run-state transitions and progress values should reflect actual backend state, not fixed browser-side values. The implementation supports concurrent processing of multiple agent records with bounded concurrency.

4.3.8 Result and Exception Module

Completed result data is stored under the results/ directory as JSON files, according to the supplied project notes. The application provides endpoints to retrieve a run's results, the latest completed result, and unresolved exceptions. The latest-results logic was corrected to select persisted files newest-first after an older result was sometimes returned.

4.3.9 Evaluation Module

The evaluation module compares matching output with a held-out ground-truth mapping. The ground-truth mapping is isolated from the matching pipeline and is used only for evaluation. Metrics include overall match rate, precision, recall, fast-path versus agent resolution, accuracy by transaction pattern, single-agent versus proposer-verifier performance, latency, and throughput.

4.4 Data Persistence Design

The supplied architecture does not document a relational database. It uses CSV input and JSON result persistence, supported by directories for uploads, logs, run state, and completed results. The following table describes logical records and artifacts; it is not a claim that these are relational database tables.

Record or Artifact	**Important Attributes**	**Purpose**
Source transaction	Source record id, source type, reference, amount, date, description, source fields	Represents a record from a bank, ledger, or gateway export
Candidate match	Target record ID, candidate record ID, amount difference, date difference, reference comparison, status	Stores evidence used to evaluate a possible match
Settlement combination	Target settlement ID, component record IDs, combined amount, fee tolerance, consistency checks	Represents a many-to-one settlement hypothesis
Reconciliation run	Run ID, creation time, status, progress, input references, configuration	Tracks an asynchronous reconciliation job
Match result	Run ID, source record ID, matched record IDs, decision type, evidence, final status	Records a confirmed or proposed reconciliation result
Exception	Run ID, source record ID, exception reason, candidates considered, final state	Keeps unresolved or unsafe cases visible
Reasoning trace	Record ID, Proposer output, tool evidence, Verifier decision, objections, retry count	Supports investigation of an agent decision
Evaluation record	Source record ID, ground-truth mapping, predicted decision, evaluation outcome	Measures system performance without exposing labels to matching logic
Persisted result file	Run identifier, completion state, result payload	Allows completed results to be retrieved after processing

4.5 REST API Design

The following endpoints illustrate a possible API structure.

HTTP Method	Endpoint	Purpose
POST	/api/run	Starts a reconciliation run
GET	/api/status/{run_id}	Retrieves the status of a run
GET	/api/results/{run_id}	Retrieves results for a specific run
GET	/api/results/latest	Retrieves the newest completed persisted result
GET	/api/reasoning-trace/{record_id}	Retrieves a reasoning trace for a record
GET	/api/exceptions/{run_id}	Retrieves exceptions for a run

The frontend must handle unsuccessful API responses and distinguish running, completed, failed, and unavailable result states. The exact payload and authentication contracts are defined by the route implementations and runtime configuration.

The latest-results endpoint must select the newest completed result rather than relying on arbitrary file traversal order. This behavior was corrected after a stale-result problem was identified in the supplied project notes.

The current project uses file-based gateway imports. Its application API endpoints support reconciliation runs and results; they are not Razorpay payment-creation endpoints or webhook receivers.

4.6 Runtime Architecture

The confirmed local development commands start the backend with Uvicorn on port 8000 and the frontend on port 3000. The backend is launched from the backend directory, while the frontend is launched from the frontend directory. A root-level development command, pnpm run dev, is documented as starting the Next.js frontend and FastAPI backend together.

The documented runtime uses separate frontend and backend development processes. The frontend communicates with FastAPI over HTTP, while the backend manages asynchronous jobs and JSON-based result persistence. Production hosting and orchestration are outside the current implementation scope.

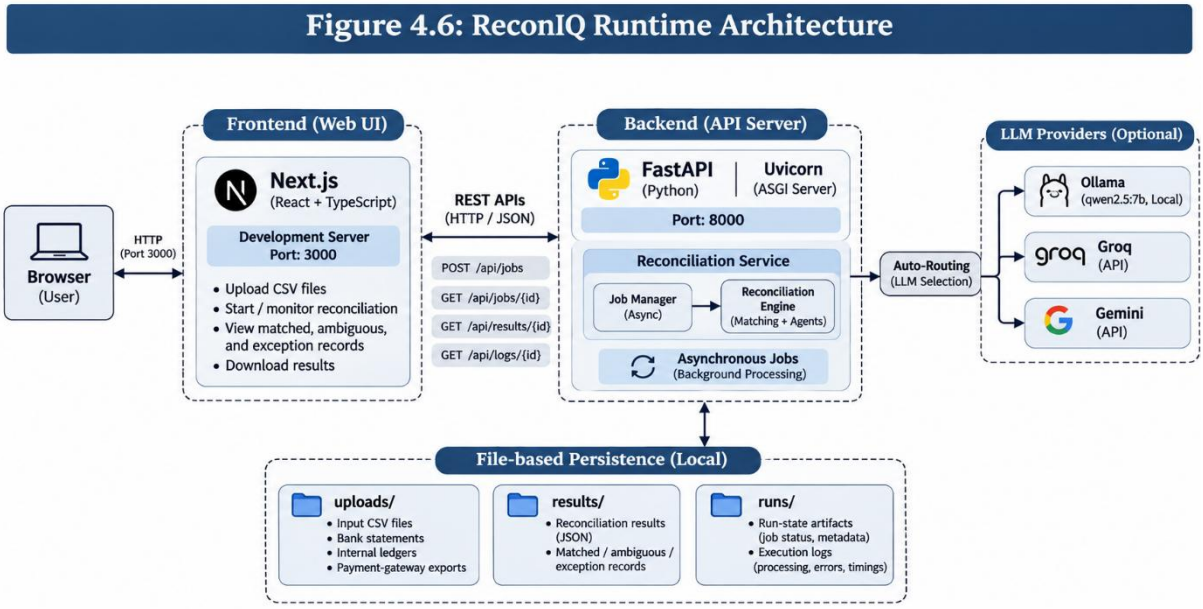


Fig 4.6 ReconIQ Runtime Architecture

4.7 Summary

ReconIQ separates deterministic matching, agent-based investigation, independent verification, asynchronous run management, persistence, and evaluation. CSV input and JSON result storage match the documented implementation. The Proposer-Verifier architecture, settlement-consistency ranking, and explicit exception handling are central design decisions. The runtime architecture reflects the current file-based input workflow and local frontend/backend execution model. Direct Razorpay API integration and production hosting are future scope.

CHAPTER-5
IMPLEMENTATION AND TESTING

5.1 Introduction

This chapter explains the implementation of the backend, frontend, file-based data persistence, agent tools, asynchronous execution, and testing.

5.2 Development Environment Setup

The documented development workflow uses Python for the backend and pnpm for the frontend. The project repository contains backend/ and frontend/ folders, test code under backend/tests/, dependency manifests, an environment template, result artifacts, logs, run state, uploaded files, and a root development script.

The documented backend setup commands are:

```
cd backend

python -m venv venv
```

On Windows:

```
venv\Scripts\activate
```

On Linux or macOS:

```
source venv/bin/activate
```

Install the dependencies:

```
pip install -r requirements.txt
```

Create `backend/.env` using `backend/.env.example` as the template. Backend configuration is supplied through environment variables and a local environment file based on `backend/.env.example`. Provider credentials must remain outside source control.

Start the backend using the documented local command:

```
uvicorn app.main:app --reload --port 8000
```

Frontend Setup

From a separate terminal:

```
cd frontend

pnpm install

pnpm run dev
```

The documented frontend address `is http://localhost:3000`. The documented backend address `is http://localhost:8000`.

Root Development Command

The repository also includes a root-level command:

```
pnpm run dev
```

The root development command `pnpm run dev` is documented as starting the Next.js frontend and FastAPI backend together.

5.3 Backend Implementation

The backend is implemented in Python using FastAPI. It accepts reconciliation-run requests, starts asynchronous execution, reports status, returns persisted results, provides record-level reasoning traces, and retrieves exception lists. The project layout separates data generation, matching, agents, API routes, evaluation, and tests.

The documented repository structure includes:

```
ReconIQ/

├── backend/

|   ├── app/

|   |   ├── data_generation/

|   |   ├── matcher/

|   |   ├── agent/

|   |   ├── api/
```

```
│   │   └── eval/
│   ├── tests/
│   ├── requirements.txt
│   └── .env.example
├── frontend/
│   ├── app/
│   ├── components/
│   ├── lib/
│   ├── scripts/
│   └── package.json
├── docs/
│   ├── architecture.png
│   └── proposer-verifier.png
├── results/
├── logs/
├── run_state/
├── uploads/
├── package.json
└── README.md
```

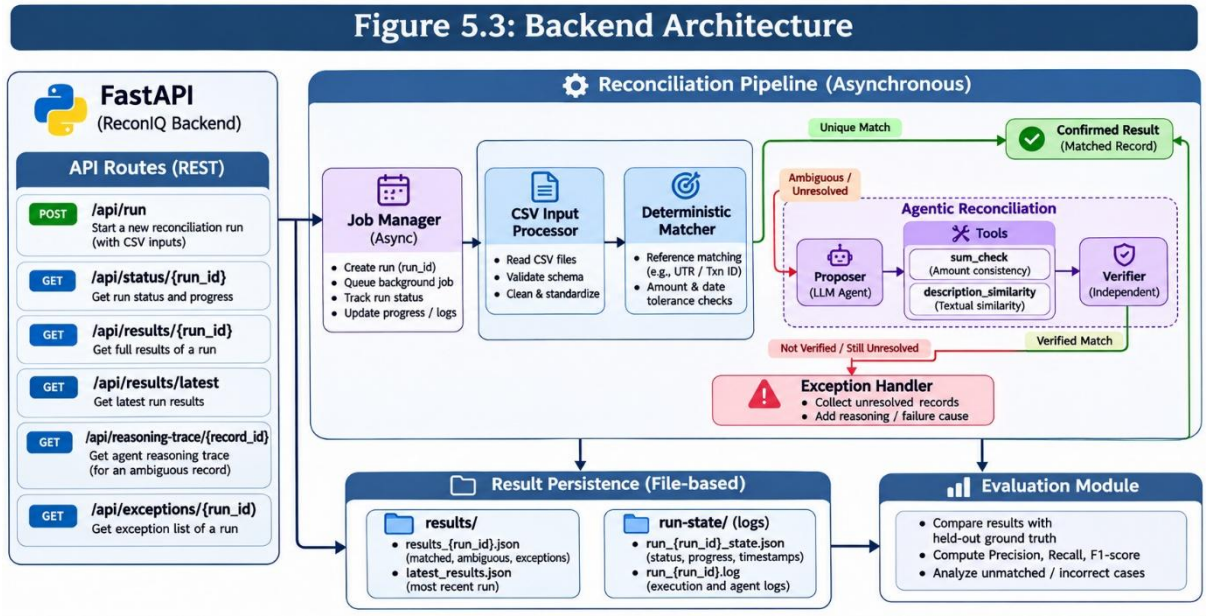


Fig 5.3 Backend Architecture

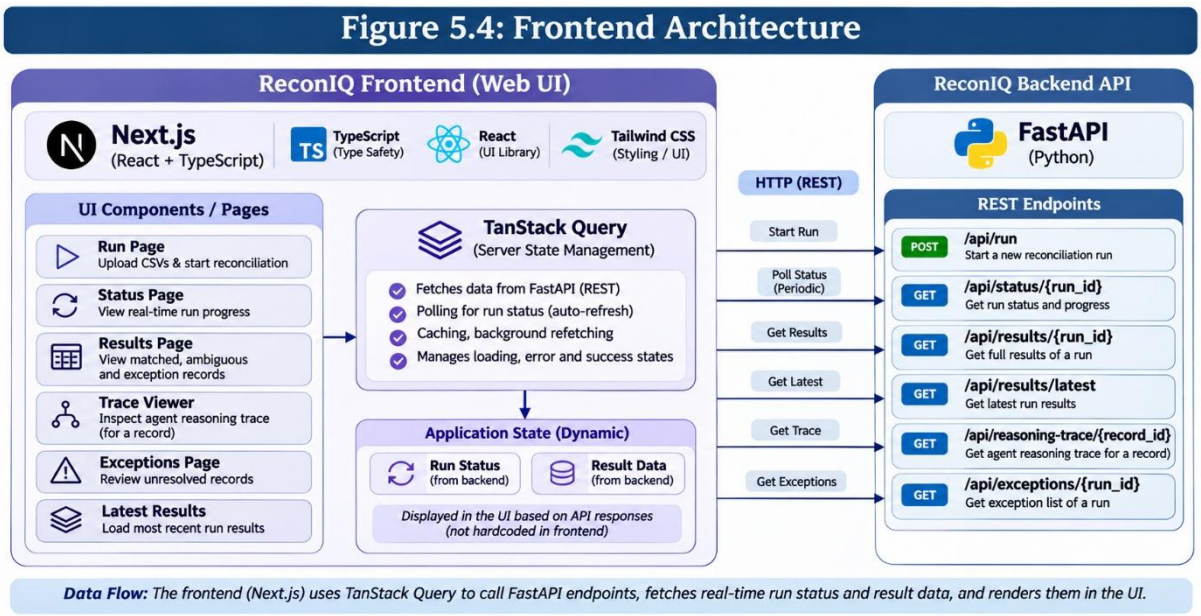
5.4 Frontend Implementation

The frontend uses Next.js, React, TypeScript, TanStack Query, and Tailwind CSS. The backend supplies the data displayed by the interface. The frontend polls the status endpoint while a run is in progress and loads the most recently completed persisted result through /api/results/latest for the Runs control room.

The documented frontend behavior includes:

- Initiating or accessing reconciliation runs through the backend API.
- Polling actual backend status to display live progress.
- Loading persisted results after the backend completes a run.
- Accessing exception data for a run.
- Retrieving reasoning traces for selected records.
- Representing backend errors and unavailable results rather than using fabricated browser-side progress.

The frontend provides access to the Runs control room, status updates, persisted results, reasoning traces, and exception data. It uses backend responses as the source of run state rather than simulated browser-side progress.



5.4 Frontend Architecture

5.5 File-Based Data Persistence

The supplied implementation uses CSV files for input and JSON files for completed results. It does not document a relational database. Therefore, this section describes file-based persistence rather than inventing a database engine or SQL schema.

The documented directories include:

- `uploads/` for uploaded input files.
- `results/` for completed JSON result files.
- `run_state/` for run-state artifacts.
- `logs/` for execution logs.

The latest-results endpoint reads persisted results and selects the newest completed result. A bug that caused an older file to be returned was corrected by selecting results newest-first. This behaviour is particularly important when users execute multiple runs in succession.

Persistence checks should verify that completed results are valid JSON, that a result can be retrieved by run identifier, and that /api/results/latest selects the newest completed result.

The project notes document a correction to newest-first selection; formal retention and backup procedures are not described in the repository.

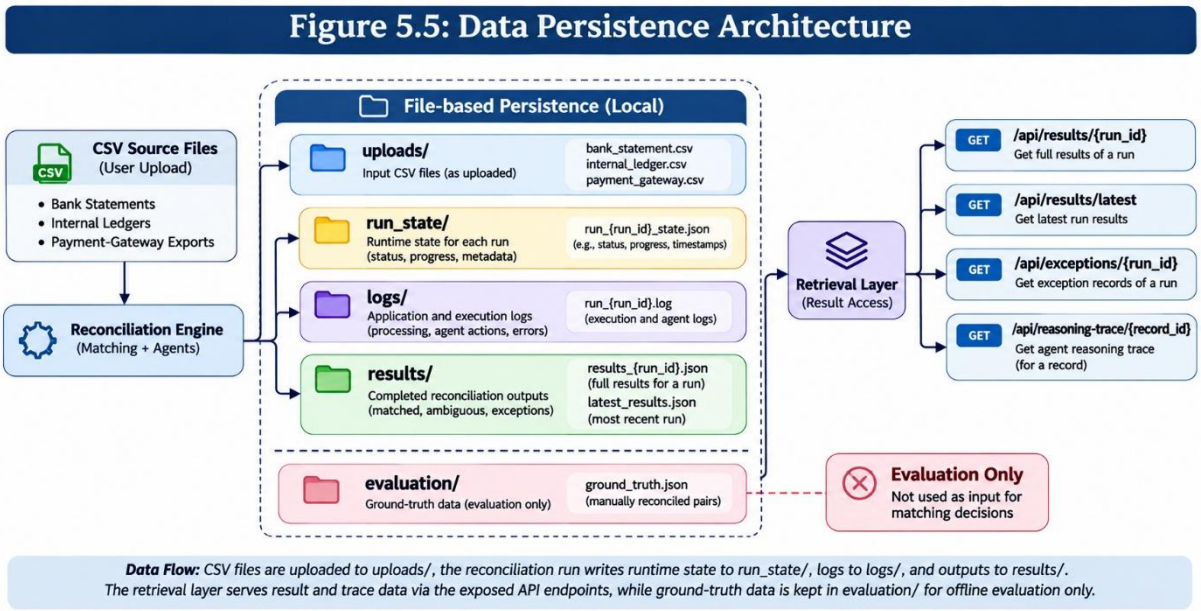


Fig 5.5 Data Persistence Architecture

5.6 Agent Tools Implementation

The agent tools provide focused operations that support investigation of ambiguous reconciliation records.

sum_check

The `sum_check` tool evaluates many-to-one settlement candidates. It searches combinations of ledger records and checks whether their total explains a target settlement amount within the configured fee tolerance. The project notes document a cross-batch error in which an incorrect combination was numerically closer to the target than the correct combination. Candidate ranking was changed to prioritize settlement consistency before numerical closeness.

description_similarity

The `description_similarity` tool compares descriptions that may differ between source systems. Similarity provides supporting evidence but is not used as an absolute veto, because a genuine transaction may have a very different description in another system.

Tool-call and failure handling

The implementation notes document provider-specific tool-call and structured-output failures, empty model decisions, token pressure, and rate limiting. The system uses explicit terminal decision tools, bounded retries, provider-specific recovery, smaller candidate payloads, tighter turn limits, and throttled concurrency to address these failure modes.

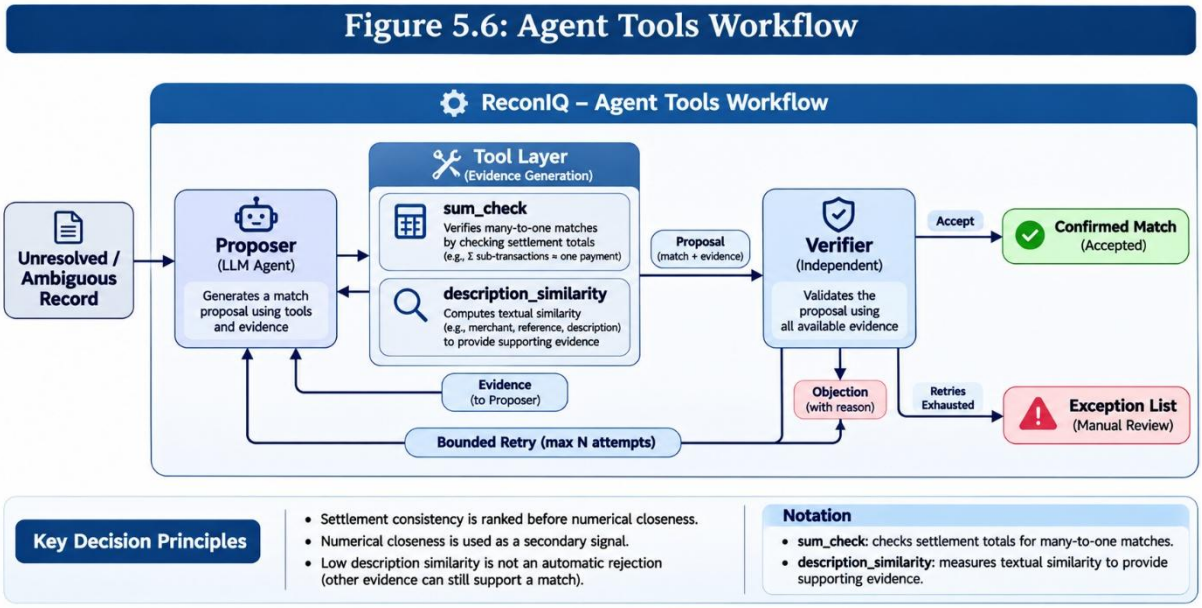


Fig 5.6 Agent Tools Workflow

5.7 Asynchronous Processing and Provider Routing

Agent-based resolution can take longer than deterministic matching, particularly when a provider requires multiple tool calls or a verification retry. The backend therefore runs reconciliation as an asynchronous job. The frontend polls the backend status endpoint and displays runtime progress based on actual backend state.

The documented provider modes are local, Groq, gemini, and auto. Local inference uses Ollama with qwen2.5:7b. Cloud modes use tool-calling interfaces that allow the agent workflow to use the same tools. The auto mode supports provider-aware routing when rate limits or provider availability become constraints.

Concurrent agent records are handled with bounded concurrency to reduce the risk of overwhelming local compute or provider limits. Blocking work is moved away from the FastAPI event loop so status polling can continue during reconciliation.

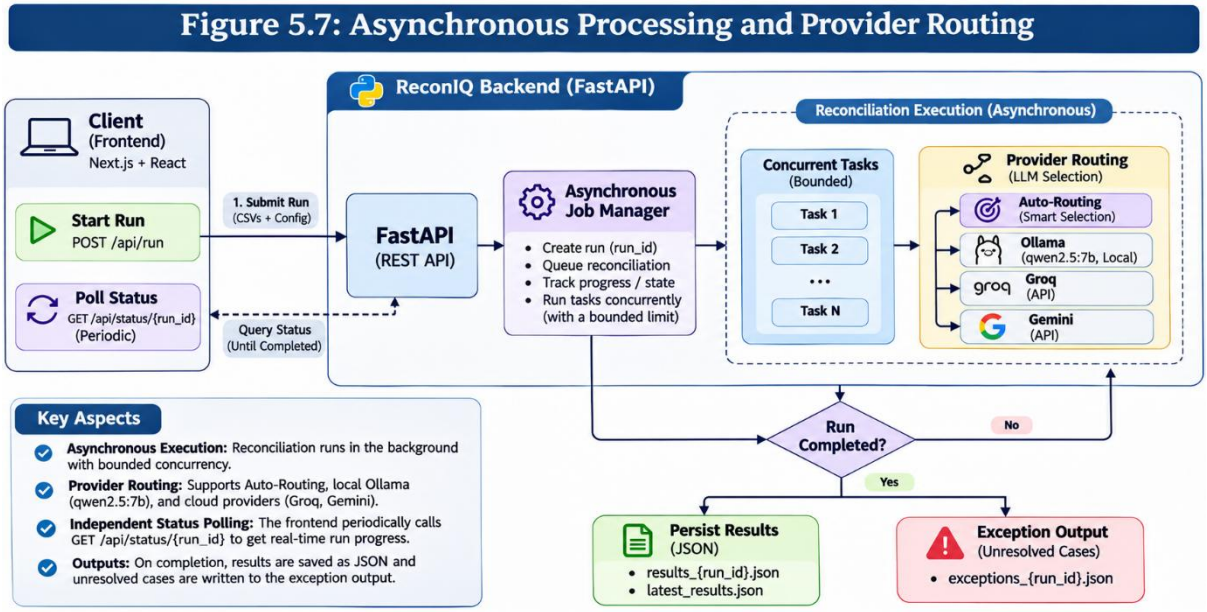


Fig 5.7 Asynchronous Processing and Provider Routing

5.8 Testing Strategy

Testing should establish both matching correctness and system behaviour. The supplied notes state that the backend test suite covers reference matching, tolerance matching, amount sanity checks, subset matching, and description similarity. The following testing categories provide a framework for documenting the extent and outcome of that work.

Unit Testing: Tests individual matcher rules and tools, including reference matching, amount consistency, tolerance evaluation, settlement subset ranking, and description similarity.

Integration Testing: Tests the path from a run request through input processing, matching, agent resolution, result persistence, and retrieval.

API Testing: Tests documented endpoints, required request fields, invalid run identifiers, incomplete jobs, completed jobs, exception retrieval, and latest-result behaviour.

Asynchronous Processing Testing: Confirms that status polling continues while a reconciliation job runs, concurrent processing respects configured limits, and the job reports a terminal state only after its result is persisted.

Agent Workflow Testing: Tests Proposer output validation, independent verification, objections, bounded retries, empty responses, provider failures, and the exception path when no defensible conclusion is available.

Settlement Testing: Tests legitimate many-to-one combinations and planted incorrect combinations, including the documented cross-batch near-miss case.

Evaluation Testing: Confirms that ground-truth labels remain isolated from the runtime matcher and that metrics are computed against the held-out mapping.

Frontend Testing: Checks that the UI polls real status, loads persisted results, displays exceptions, and handles API failures.

Provider and Reliability Testing: Tests cloud API failures, rate limits, invalid tool-call structures, and provider-specific response recovery. Provider tests must not expose real API credentials.

5.9 Sample Test Cases

Test ID	Test Scenario	Expected Result or Recorded Evidence
TC01	A record has a reliable reference match and a consistent amount.	The deterministic matcher identifies the candidate for confirmation. Reference matching is included in the backend test suite.
TC02	A reference match has a suspicious amount difference.	The record is flagged rather than blindly confirmed. Amount sanity checks are included in the backend test suite.
TC03	A record has exactly one candidate within the configured amount and date tolerances.	The unique qualifying candidate is selected. Tolerance matching is included in the backend test suite.
TC04	Multiple candidates fall within the configured tolerance.	The record is treated as ambiguous and escalated rather than automatically confirmed.
TC05	No candidate meets the matching rules.	The record remains unresolved or enters the agent workflow.
TC06	A legitimate transaction has a substantially different description but consistent financial evidence.	Low description similarity does not automatically reject the candidate. This behavior is documented in proof case BANK_0042.
TC07	Four ledger records together explain a many-to-one settlement.	The settlement tool evaluates the combination. Subset matching is included in the backend test suite.
TC08	An incorrect cross-batch combination is numerically closer than the correct settlement.	Ranking prioritizes settlement consistency before numerical closeness. This correction is documented in proof case BANK_0028.
TC09	A description-mismatch case contains an unlabeled decoy.	The correct candidate is not rejected solely because of low textual similarity. This case is documented as BANK_0042.

Test ID	Test Scenario	Expected Result or Recorded Evidence
TC10	A model returns an empty terminal decision.	The response is handled as a failure or retried, rather than accepted as a valid match. Explicit failure handling is documented in the project notes.
TC11	A provider returns a tool-call or structured-output error.	Bounded retry or provider-specific recovery is applied. This failure class and its fixes are documented in the project notes.
TC12	A run is started through /api/run.	The API starts a reconciliation job and returns a run identifier.
TC13	The frontend polls status while a job is running.	Status reflects backend runtime state while the job continues asynchronously.
TC14	Several completed result files exist.	/api/results/latest selects the newest completed result. The correction to newest-first selection is documented in the project notes.
TC15	An exception is generated during a run.	/api/exceptions/{run_id} exposes the exception data for that run.
TC16	A reasoning trace is requested for a record.	The API returns the requested trace or an appropriate not-found response.
TC17	Ground-truth data is available to the evaluation layer.	The expected mapping remains separate from the matching pipeline and is used for evaluation.
TC18	A gateway export is imported through the supported CSV workflow.	Input compatible with the supported schema can be processed; unsupported or malformed input should be reported rather than silently accepted.

5.10 Performance and Deployment Evaluation

Performance evaluation should include:

- Overall match rate.
- Precision and recall against the held-out ground truth.
- Match quality by transaction pattern.
- Share of records resolved in the deterministic fast path.
- Share of records resolved through the agent workflow.
- Performance of the proposer-verifier workflow compared with a single-agent baseline.
- Average and percentile latency per record and per batch.
- Throughput and completion time for a batch of known size.
- Model call count, token usage, provider errors, and rate-limit events.
- CPU, memory, and, where relevant, GPU usage.

- Persistence latency and successful retrieval of completed results.
- Status-polling responsiveness during a long-running job.

A reproducible evaluation records dataset size, transaction-pattern distribution, model/provider configuration, retry limits, concurrency, hardware, network conditions, and the tested source version. A speedup value is meaningful only when its baseline and workload are clearly defined.

5.11 Summary

This chapter describes the implementation workflow and testing methodology. The combination of functional testing, security checks, container verification, and deployment evaluation helps determine whether the platform satisfies its requirements.

CHAPTER-6
RESULTS AND FUTURE ENHANCEMENTS

6.1 Introduction

This chapter summarizes the results and engineering contributions documented for ReconIQ, describes how the evaluation should be interpreted, and identifies limitations and possible future work. Results from separate runs are presented independently because they use different configurations or evaluation contexts.

6.2 Expected Results

The expected outcomes of ReconIQ are:

1. Reconcile transaction records from bank statements, internal ledgers, and payment-gateway exports.
2. Confirm straightforward matches through deterministic rules without unnecessary model calls.
3. Flag suspicious amount mismatches even when references appear to agree.
4. Identify unique candidates using configurable amount and date tolerances.
5. Keep ambiguous and unresolved records out of the automatically confirmed set.
6. Use the Proposer-Verifier workflow to investigate genuinely ambiguous cases.
7. Evaluate many-to-one settlement combinations using consistency-aware ranking.
8. Use description similarity as supporting evidence rather than an absolute decision rule.
9. Run reconciliation asynchronously and provide observable status through API polling.
10. Persist completed results and enable retrieval of run outputs, exceptions, and reasoning traces.
11. Evaluate system behavior against a held-out ground-truth mapping.
12. Provide a clear record of unresolved exceptions and known failure modes.

These are system objectives. Their degree of achievement should be supported by test evidence and metrics rather than assumed from the presence of the relevant code or configuration.

6.3 Result Evaluation

The evaluation compares reconciliation output against a held-out ground-truth mapping. The project notes report two separate runs. Their metrics measure different aspects and must not be combined into one result.

Evaluation Run	Metric	Reported Result	Interpretation
Earlier full benchmark	Precision	Approximately 95.5%	The reported share of confirmed matches that were correct under the tested configuration.
Earlier full benchmark	Sequential speedup	Approximately 3.85×	The measured speedup against the sequential comparison baseline used for that benchmark.
Separate completed run	Batch size	64 records	A later run used a 64-record batch.
Separate completed run	Overall match rate	71.88%	The proportion of the 64-record batch receiving a match in that run.
Separate completed run	Durable-results retrieval	Reported as verified	The run confirmed the persisted-results path used by the latest-results control room.

The reported 95.5% precision and 3.85× speedup belong to the earlier benchmark. The 71.88% match rate belongs to the separate 64-record run. They are not competing estimates of the same metric. Further metrics, including per-pattern precision and recall, fast-path versus agent-path quality, latency, throughput, and provider error rates, are not quantified in the supplied project notes and are therefore not assigned invented values here.

6.4 Project Contributions

The documented project contributions include:

- Combining deterministic record matching with AI-assisted investigation only for ambiguous or unresolved cases.

- Checking amount consistency for reference-based candidate matches.
- Applying unique-candidate rules to tolerance-based matching.
- Adding independent verification to challenge an agent's proposed decision.
- Supporting many-to-one settlement analysis using sum_check.
- Ranking settlement candidates by consistency before numerical closeness.
- Treating description similarity as supporting evidence rather than a strict rejection rule.
- Keeping ground-truth mapping separate from the matching pipeline.
- Supporting asynchronous reconciliation, runtime status polling, and persisted JSON output.
- Providing APIs for run results, latest results, record-level reasoning traces, and exception lists.
- Resolving the documented stale-latest-result selection issue.
- Addressing empty model decisions, provider-specific tool-call errors, excess payload/token usage, rate-limit pressure, and event-loop blocking through explicit handling and bounded execution strategies.

These contributions are based on the supplied technical notes. The final submission should connect each contribution to source code, test output, or benchmark artifacts wherever possible.

6.5 Limitations

The current information and documented architecture have several limitations that should be acknowledged.

- Razorpay integration is file-based. The current workflow uses exported transaction or settlement data supplied through the application's CSV input path; live checkout, payment creation, webhook processing, and payment-status API calls are outside the confirmed scope.
- Input compatibility depends on the source export matching the supported CSV schema and normalization rules.
- Financial matching remains sensitive to incomplete references, currency differences, inconsistent timestamps, settlement fees, and missing source data.
- The deterministic tolerance values may require adjustment for different institutions, gateways, currencies, or settlement rules.

6.6 Future Enhancements

6.6.1 Direct Razorpay API and Webhook Integration

If the intended scope includes direct Razorpay integration, a future phase can implement the specific payment API or webhook workflow required by the product. The implementation should define the business purpose, configure test-mode credentials, verify webhook signatures where applicable, validate payment states, handle duplicate event delivery, and record the source of each imported transaction. Integration should be described as completed only after tests demonstrate the exact flow. Until then, export-based reconciliation remains the confirmed scope.

6.6.2 Human-in-the-Loop Exception Review

A review interface could allow authorized reviewers to inspect candidates, compare supporting values, view reasoning traces, record a decision, and provide an explicit resolution reason. Review decisions should be auditable and should not overwrite the original model proposal without preserving the change history.

6.6.3 Scalable Processing Architecture

As batch sizes grow, run scheduling, matching, tool calls, and persistence can be separated into independently managed components if measurement justifies that complexity. Any queue or worker system should preserve idempotent processing, bounded concurrency, status consistency, and result traceability.

6.6.4 CI/CD Automation

A continuous-integration pipeline could run unit and integration tests, validate schemas, lint code, build the frontend, and perform backend test suites before packaging a release. If containers are adopted, the pipeline could build and scan images and publish them through a controlled release process.

6.6.5 Monitoring and Observability

Structured logs, run-level metrics, tracing, and dashboards could show job duration, records per second, fast-path share, agent escalation rate, verification disagreement rate, provider

failure count, and exception frequency. Monitoring should avoid logging raw financial or personally identifiable information unnecessarily.

6.6.6 Cloud Deployment

The system could be deployed in a managed environment after operational requirements are specified. Any future deployment beyond the documented local workflow should define secrets management, persistent storage, resource limits, health checks, backups, rollback, and access controls. Availability or autoscaling claims must be based on verified configuration and tests.

6.6.7 Audit, Access Control, and Data Governance

Future work can formalize authentication, authorization, least-privilege access, retention rules, deletion workflows, encryption, audit-log immutability, provider data-handling rules, and redaction of sensitive fields before model calls. These controls are particularly important if real financial records are processed.

6.6.8 Feedback-Driven Matching Improvements

Verified reviewer decisions could help identify recurring source-system patterns and improve deterministic rules. Any feedback-driven process should prevent evaluation leakage and distinguish previously seen cases from independent test cases. Automated rule changes should be evaluated on a held-out dataset before being applied to production matching.

6.6.9 Advanced Anomaly Detection and Reconciliation Analytics

Future versions could analyze recurring mismatches, duplicate settlements, abnormal fee amounts, unusual timing, and source-specific drift. Such capabilities should produce inspectable alerts and evidence rather than silently changing final match decisions. Their usefulness should be measured against labeled anomalies and reviewer feedback.

CONCLUSION

ReconIQ presents a hybrid approach to financial reconciliation across bank statements, internal ledgers, and payment-gateway exports. It uses deterministic matching for cases with clear reference, amount, and date evidence. When a single defensible candidate cannot be identified, an agentic workflow can investigate the case through a Proposer and an independent Verifier. The system preserves unresolved cases as exceptions rather than forcing uncertain matches.

The documented implementation uses a Python and FastAPI backend, asynchronous job execution, CSV input, JSON result persistence, and a Next.js frontend built with React, TypeScript, TanStack Query, and Tailwind CSS. The agent layer provides the sum_check tool for settlement combinations and description_similarity as supporting semantic evidence. The API exposes run initiation, status, completed results, latest results, reasoning traces, and exceptions. The supplied notes also record engineering corrections addressing decoy leakage, settlement-combination ranking, empty model decisions, provider-specific failures, token and rate-limit pressure, event-loop blocking, and stale latest-result selection.

The available evaluation notes report approximately 95.5% precision and approximately 3.85 times sequential speedup in an earlier benchmark. A separate 64-record run recorded a 71.88% overall match rate and verified the durable-results retrieval path. Because these values come from separate runs, the report treats them independently. Further benchmark details, execution logs, and environment settings should be attached so the results can be reproduced and assessed accurately.

The central contribution of ReconIQ is its conservative treatment of uncertainty. Deterministic rules handle straightforward cases, agent investigation is reserved for ambiguity, an independent verification stage challenges proposed decisions, and cases that remain unclear can be recorded as exceptions. Razorpay integration is file-based through exported records and the supported CSV input workflow. Live checkout or webhook integration is not part of the current implementation.

REFERENCES

[1] Razorpay, “API Reference Guide and Payment Gateway Integration Documentation.” [Online]. Available: https://razorpay.com/docs/

[2] FastAPI, “FastAPI Documentation.” [Online]. Available: https://fastapi.tiangolo.com/ .

[3] Vercel, “Next.js Documentation.” [Online]. Available: https://nextjs.org/docs .

[4] Meta Platforms, Inc., “React Documentation.” [Online]. Available: https://react.dev/learn

[5] Microsoft, “TypeScript Documentation.” [Online]. Available: https://www.typescriptlang.org/docs/ .

[6] TanStack, “TanStack Query Documentation for React.” [Online]. Available: https://tanstack.com/query/latest/docs/framework/react/overview .

[7] Ollama, “Tool Calling.” [Online]. Available: https://github.com/ollama/ollama/blob/main/docs/capabilities/tool-calling.mdx .

[8] Tailwind CSS, “Tailwind CSS Documentation.” [Online]. Available: https://tailwindcss.com/docs .

[9] Python Software Foundation, “Python Documentation.” [Online]. Available: https://docs.python.org/3/.

[10] Groq, “API Reference and Tool Use Documentation.” [Online]. Available: https://console.groq.com/docs/api-reference and https://console.groq.com/docs/tool-use/overview.

[11] Google, “Gemini API Documentation.” [Online]. Available: https://ai.google.dev/gemini-api/docs .

[12] Qwen, “Qwen2.5-7B-Instruct Model Card.” [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-7B-Instruct.